JOBHELPER

JobHelper technical documentation

How the app is built, run and tested

For JobHelper on macOS
1 figure · 14 tables

Contents

1. Overview **5**

Related documents 6

2. Install the app **7**

Prerequisites 7

Install the packages 7

3. Run the app **9**

Start the window 9

Run a search without the window 9

4. Use the command line **10**

Protect and restore the data 10

5. Understand the structure of the code **12**

The agents 12

The modules around the agents 12

How the window talks to the logic 13

6. Work with the data files **15**

7. Test a change **16**

Run the tests 16

What the tests may never touch 16

Test the window 16

8. Update the documentation **17**

Take the pictures of the user guide again 17

Keep the numbers of figures and tables in order 17

Build the PDFs 17

Write in the house style 17

9. Troubleshoot problems **19**

List of figures

Figure 1. The five agents and what each keeps 5

List of tables

Table 1. The rules every change must keep 6

Table 2. The documents of the project 6

Table 3. The main packages 8

Table 4. The most used options of run.py 9

Table 5. The commands of status.py 10

Table 6. The actions of status.py security 11

Table 7. The folders of the project 12

Table 8. The five agents 12

Table 9. The modules around the agents 13

Table 10. The files and folders in data 15

Table 11. What the tests never touch 16

Table 12. The page rules of the PDFs 17

Table 13. The rules for writing the documentation 18

Table 14. Problems while working on the code 19

Applies to: the JobHelper source code, on macOS with Python 3.10 or newer.

This document is for people who work with the code: it explains how to install the app, how to run it from the command line, how the code is laid out, and how to test and document a change. To use the app, read the user guide, GUIDE.md.

1. Overview

JobHelper is a desktop app for macOS that finds job postings, scores them against a CV, writes an adapted CV and a cover letter for the good ones, and tracks applications and replies. It is written in Python. The window is built with Qt Quick (PySide6). All model calls go through the Claude Code program, so that they use the person's Claude subscription and never the paid API.

The work is done by five agents (Figure 1). A search runs the first four in order; an email scan runs the fifth.

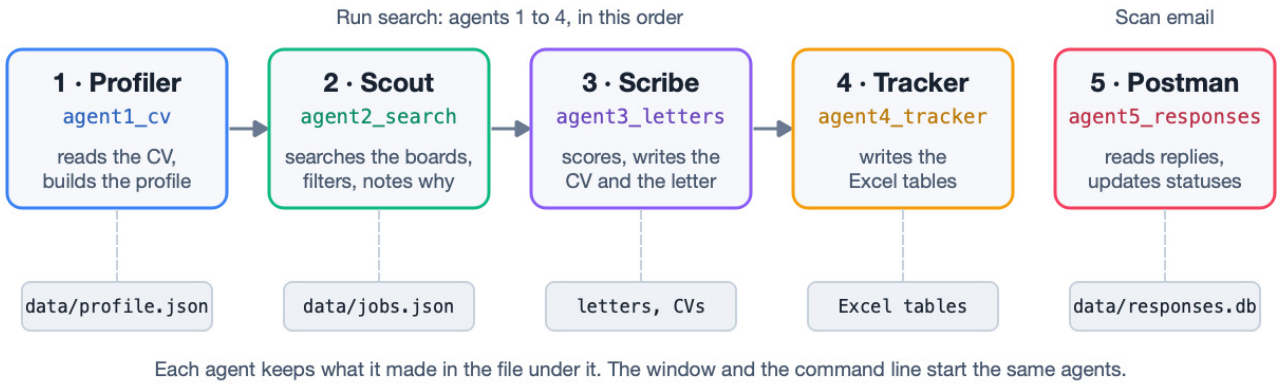


Figure 1. The five agents and what each keeps

Six rules shape the whole code base, and every change must keep them (Table 1).

Table 1. The rules every change must keep

Rule	What it means in the code
The subscription, never the paid API	llm.ask refuses to run while ANTHROPIC_API_KEY is set. No code path calls the API directly
The mailbox is read-only	Folders are opened read-only and messages are fetched with PEEK. Nothing sends, deletes, moves or marks mail
Nothing is claimed that the CV does not show	Every written text is checked in plain code against the CV after the model wrote it
The person's own letters are never changed	Only letters the app wrote may be replaced or moved
Nothing is deleted	Data is backed up before a bulk change; documents are moved to an encrypted backup, old copies to the Trash
No personal data in the code	Names and facts of a real person never appear in code, tests or documents

Related documents

Table 2 lists the other documents of the project.

Table 2. The documents of the project

Document	Use it for
GUIDE.md	Using the app, for someone who never opens a terminal
COMMANDS.md	The full reference of every command and option
HANDBOOK.md	The rules the app is built by, decisions already made, what is left to do
CHANGELOG.md	What changed, day by day. The app shows it under **Help**

2. Install the app

Prerequisites

Before you install, make sure you have:

- macOS.
- Python 3.10 or newer.
- The Claude Code program, signed in with a Claude subscription: run claude auth login once.
- Optional: LibreOffice, for PDF copies of the documents.

> **Important**
> Do not set ANTHROPIC_API_KEY. Leaving it unset is what makes the app use the subscription, and the app refuses to run a search while it is set.

Install the packages

1. Open a terminal in the project folder.
2. Create a virtual environment and turn it on:

   ```
   python3 -m venv .venv
   source .venv/bin/activate
   ```

3. Install the exact, hash-checked packages:

   ```
   pip install --require-hashes -r requirements.lock
   ```

4. Optional: copy the example settings file. The mailbox password never goes into it.

   ```
   cp .env.example .env
   ```

Table 3 lists the main packages and what each is for.

Table 3. The main packages

Package	Used for
claude-agent-sdk	Driving the Claude Code program, which uses the subscription
PySide6	The window (Qt Quick and QML) and the PDF of the documentation
python-docx, pypdf	Reading a CV; writing CVs and letters as Word files
openpyxl	The Excel tables
requests, beautifulsoup4, lxml	Reading the job boards
cryptography	AES-256-GCM encryption of the data files and the backups
pytest	The tests. pyflakes, installed separately with pip install pyflakes, checks for unused or undefined names

3. Run the app

Start the window

1. Turn the virtual environment on.
2. Start the app:

   ```
   python app_qt.py
   ```

3. Follow the welcome card: sign in to Claude, choose a CV, and connect a mailbox if you want replies to be read. The user guide describes each step.

Run a search without the window

A search can also be run from the command line. It does the same work as **Run search** and writes to the same files.

```
python run.py
```

Table 4 lists the options used most often.

Table 4. The most used options of run.py

Option	What it does
--where CITY	Searches in another city than the one saved in Options
--source NAME	Chooses the job boards: czech, linkedin, remote, adzuna, jooble or all
--include-remote	Adds the remote boards
--threshold N	Writes documents only from this score
--per-query N	Asks each board for this many postings per role
--all-languages	Keeps postings written in other languages than English
--max-age-days N	Drops postings older than this
--reuse-profile	Does not read the CV again
--skip-search	Scores the postings already in the list, without searching

> **Note**
> Do not run a search from the command line while the window is running one. Both write the same job list.

4. Use the command line

Everything the window does with applications and replies can also be done with status.py. Table 5 lists its commands.

Table 5. The commands of status.py

Command	What it does
python status.py list	Lists every tracked job with its id and status
python status.py apply ID	Marks a new job as applied
python status.py update ID STATUS	Sets the status of a job
python status.py sync	Reads replies in the mailbox and updates statuses. --days N chooses how far back
python status.py responses	Shows the numbers of replies and rejection reasons
python status.py followups	Lists applications with no reply for a number of days
python status.py rebuild	Writes the Excel tables again
python status.py retailor ID	Writes a letter and an adapted CV for one job
python status.py pdf	Makes PDF copies of the documents (needs LibreOffice)
python status.py letters	Takes in letters the person wrote themselves
python status.py usage	Shows the Claude usage of the app, in total and per kind of work
python status.py security ACTION	Encryption, Keychain and backup tools (Table 6)

Protect and restore the data

The data files are encrypted with a key kept in the macOS Keychain. Table 6 lists the actions of python status.py security.

Table 6. The actions of status.py security

Action	What it does
status	Says whether encryption is on and whether a watched file changed
recovery-key	Prints the recovery key. The window shows the same key in the **Account** card
restore-key	Puts a saved recovery key back into the Keychain
approve	Accepts a change to a rule file, the letter template or the master CV as trusted
backup, backup-all	Makes an encrypted backup of the data, or of everything
restore, test-restore	Restores a backup, or proves that a backup can be restored

Warning
Anyone who has the recovery key can read the data. Never print it into a log, a chat or a file that is kept.

5. Understand the structure of the code

The code is in three places (Table 7).

Table 7. The folders of the project

Folder	What is in it
jobhelper/	All the logic: the agents, the rules, reading and writing files
ui_qt/	The window: one QML file per window or part of the main screen
tests/	The tests, and the script that takes the pictures of the user guide
scripts/	Tools for the documentation: the PDF builder, the numbering check, the diagram
docs/	The user guide, this document, their PDFs and the pictures
data/	The person's data. Never part of the code; see section 6

The agents

Table 8 names the module of each agent and what it may ask Claude.

Table 8. The five agents

Agent	Module	What it does	Asks Claude
Profiler	agent1_cv	Reads the CV and builds the profile and the search roles	Once per CV
Scout	agent2_search	Searches the boards, filters, keeps every dropped posting with its reason	Never
Scribe	agent3_letters	Scores postings five per call; writes the adapted CV and the letter	Yes
Tracker	agent4_tracker	Writes the Excel tables	Never
Postman	agent5_responses	Reads replies and files each to its application	For new replies only

The modules around the agents

Table 9 groups the other modules by what they are for.

Table 9. The modules around the agents

Group	Modules	What they do
Reading postings	scrapers_cz, scrapers_intl, scrapers_remote, scrapers_ats, scrapers_page	One module per kind of source. A site that turns programs away is left alone
Rules without a model call	filters, prefilter, language, places, keywords, salary, merge, followup, planner	Level, years, language, location, profession, pay, duplicates, reminders
Writing	cv_tailor, cover_letter, letters, form_answers, interview_prep, prep_doc, pdf	The adapted CV, the letter, form answers, the interview sheet
Mail	email_client, mailhosts	Reading mailboxes read-only; which server a mailbox is read from
Claude	llm, usage	The one place that asks Claude; counting usage and knowing the limits
Storage and safety	store, db, vault, integrity, housekeeping, health, trouble	The job list, the reply database, encryption, signed rule files, backups, the start-up check, the log of errors
The window's back end	ui_api, api_documents, api_interview, api_letters, qt_bridge, bridge_runs, bridge_jobs	What the window calls. Api holds the logic; Bridge exposes it to QML and runs long work in threads

How the window talks to the logic

The window never touches a file. Every action goes through three layers:

1. A QML file in ui_qt/ calls a slot of bridge, for example bridge.saveOptions(...).
2. The slot, in qt_bridge.py, bridge_runs.py or bridge_jobs.py, calls a method of Api.

3. The method, in ui_api.py or one of the api_*.py files, checks the input, does the work and returns a plain dictionary: {"ok": True, ...} or {"ok": False, "msg": "why"}.

Long work (a search, a scan, a rewrite) runs in a thread and answers through a signal.

> **Important**
> Every input is checked twice: in the window, where the person sees a red message next to the field, and again in the Api method, so that nothing unusable is saved whatever sent it.

6. Work with the data files

Everything a person makes with the app is in data/ (Table 10). The folder is not part of the code and is never committed.

Table 10. The files and folders in data

Path	What it holds	Encrypted
data/jobs.json	Every posting and application	Yes
data/profile.json	The profile read from the CV	Yes
data/ui_settings.json	The Options	Yes
data/responses.db	Every reply that was read	Yes
data/usage.json	The Claude usage and the last known limits	No
data/backups/	Backups, made before any bulk change	Yes
data/cv/, data/my_letters/	The master CV and the person's own letters	No, so that Word can open them
data/tailored_cvs/, data/cover_letters/	The documents the app wrote	No, so that Word can open them
data/app_trouble.log	One line for each error the app carried on after	No

Follow these rules when code changes data:

1. Read and write the job list only through store, the replies only through db, and any encrypted file only through vault.
2. Call vault.make_backup(label) before a change that touches many jobs.
3. Move a document to the encrypted backup instead of deleting it; put old copies in the Trash.
4. Do not change the data while the window is running.

7. Test a change

Run the tests

1. Run the whole suite. It takes about two minutes and needs no network:

   ```
   python -m pytest -q
   ```

2. Check for unused and undefined names:

   ```
   python -m pyflakes jobhelper tests scripts
   ```

Both must pass before a change is committed.

What the tests may never touch

The tests work in temporary folders with stand-ins. Table 11 lists what they never reach and what stands in for it.

Table 11. What the tests never touch

Never touched	What stands in
The real data/ folder and .env	Temporary folders, set by the isolate_data fixture
The macOS Keychain	An in-memory keychain
Claude	A function that returns a prepared answer
Job boards and mail servers	Functions that return prepared postings and messages
Notifications, Finder, the Trash, the browser	Functions that only record that they were called

Test the window

The window is loaded without a screen and driven by the tests in tests/test_ui_qml.py. Use the helpers of its ui fixture: ui.js(code) runs JavaScript in the window, ui.find(name) finds an item by its objectName, and ui.wait(ms) lets the window run.

> **Tip**
> After a change that can be seen, take a picture of the window and look at it. A test that reads a text does not notice that the text is not drawn.

8. Update the documentation

Each change that a person can see needs three things: a line in CHANGELOG.md, the user guide brought up to date, and its pictures taken again if a pictured window changed.

Take the pictures of the user guide again

The pictures show the real window filled with invented people and companies. A script takes them:

```
python -m pytest tests/guide_shots.py -q -o addopts=""
```

Keep the numbers of figures and tables in order

Every picture has a caption under it and every table a caption above it: a line in italics that begins with the word Figure or Table, its number and a full stop. The text refers to each by its number.

1. Give a new figure or table any number that is not used yet, for example 900.
2. Renumber the captions and every reference in one go:

   ```
   python scripts/guide_number.py
   ```

Build the PDFs

```
python scripts/guide_pdf.py
```

The script makes docs/GUIDE.pdf and docs/CODE.pdf. It lays each one out with a title page, the contents, a list of figures and a list of tables, and it keeps the page rules in Table 12. A test fails while a PDF is older than its text.

Table 12. The page rules of the PDFs

Rule	How the script keeps it
A figure is never split across two pages	A picture and its caption are one block; a block that would cross a page starts on the next one
A table stays on one page	The same, with the caption above it
A table longer than a page continues between two rows	It is cut into parts that fit, each with the caption "continued" and the heading row repeated
A heading is never left alone at the bottom of a page	A heading whose text would start on the next page moves there too

Write in the house style

Table 13 gives the rules for the text of both documents.

Table 13. The rules for writing the documentation

Element	Rule
Headings	Start with a verb and name the task: "Connect your mailbox"
Steps	Numbered, one action each, in the order they are done
Reference material	In a table, with a caption and a reference from the text
Names of buttons and labels	Bold, spelled exactly as in the app
Commands and file names	Monospace. The user guide contains no commands
Links	The link text says where it leads, never just "here"
Notes	Four kinds: Note, Tip, Important, Warning, as a quoted block that starts with the bold word

9. Troubleshoot problems

Table 14 lists the problems met most often while working on the code.

Table 14. Problems while working on the code

What you see	Cause	What to do
"Unset ANTHROPIC_API_KEY to use your subscription"	The paid key is set in the shell	Run unset ANTHROPIC_API_KEY, then start again
The window says Claude is not signed in	The Claude Code program has no session	Run claude auth login
A test of a PDF fails after a change to a document	The PDF is older than its text	Run python scripts/guide_pdf.py
A test of the numbering fails	A figure or table was added or moved	Run python scripts/guide_number.py
A QML property named onSomething does nothing	QML reads such a name as a signal handler	Give the property another name
An item made by a Repeater is not found in a test	It is not a child in the object tree	Use ui.seen(part) or ui.drawn(name)
The window stutters while a thread works	Python runs one thread at a time	Give the thread short rests, as keywords.read_ahead does
Data looks wrong after an experiment	A change was made to the real data	Restore the newest backup: python status.py security restore